

120. Le record, gardé dans la cartouche

Le meilleur score survit à l'extinction : la cartouche a une petite mémoire à pile.

1. Ce qu'il faut comprendre

Toutes les variables vivent dans la mémoire de la console, qui s'efface quand on l'éteint. Un record qui disparaît à chaque extinction n'en est pas un.

Certaines cartouches portent une **pile** et une petite mémoire à elles. **Ce qui est nouveau ici** : `sauver(numero, valeur)` et `sauvegarde(numero)`. `sauver(1, record)` écrit dans la case n° 1 de cette mémoire ; `sauvegarde(1)` la relit, même des mois plus tard.

La toute première fois, cette mémoire contient **n'importe quoi**. On ne peut pas faire confiance à la case 1 : elle vaut peut-être 173, un record que personne n'a fait. D'où la **marque** : dans la case 0, le jeu écrit un nombre à lui, 42. Au démarrage, s'il ne retrouve pas 42, la mémoire est neuve : il y écrit la marque et un record de 0.

Le record n'est comparé qu'à **un seul** endroit : en entrant dans l'état `PERDU`, dans `changer()`. La leçon d'avant y avait mis le dessin de l'écran de fin ; c'est aussi le bon moment pour dire « nouveau record! » et le sauver. Voilà pourquoi une seule fonction change d'état.

On ne sauve **que** si le score bat le record : écrire dans cette mémoire est plus lent, et elle s'use un peu à chaque écriture.

À toi : garde aussi le nombre de parties jouées, dans la case 2.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <ecran>` → [27.1. La fonction ecran\(\) — éteindre et rallumer l'écran](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <sauver>` → [119.1. La fonction sauver\(\) — garder un nombre, même éteint](#)
- `#include <sauvegarde>` → [119.2. La fonction sauvegarde\(\) — relire un nombre gardé](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1510 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      71 o ← la console, à l'allumage - la mise en route de la console
appuiStart()   40 o ← titre(), perdu() - ta fonction
changer()      294 o ← titre(), jeu(), perdu(), main() - ta fonction
titre()        15 o ← main() - ta fonction
jeu()          97 o ← main() - ta fonction
perdu()        14 o ← main() - ta fonction
main()        151 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
```

AttendreImage	28 o	← image()	– attend l'image suivante	
AttendreVBlank	16 o	← ecran()	– attend que l'écran soit libre	
AttendreAcces	12 o	← EcrireTexte, EcrireNombre	– attend de pouvoir écrire une case	
AdresseCase	10 o	← texte()	– trouve l'adresse d'une case de l'écran	
EcrireTexte	10 o	← texte()	– écrit un texte, lettre par lettre	
EcrireNombre	33 o	← nombre()	– écrit un nombre en chiffres	
CopierTuiles	85 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo	
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran	
LireManette	37 o	← bouton()	– lit les huit boutons	
Tuiles	464 o	← CopierTuiles	– 29 lettres sur 43	
"	...	20 o	← texte()	– un texte gravé
"LE JEU DU A"	11 o	← texte()	– un texte gravé	
"RECORD"	6 o	← texte()	– un texte gravé	
"START: JOUER"	12 o	← texte()	– un texte gravé	
"APPUIE SUR A!"	13 o	← texte()	– un texte gravé	
"POINTS"	6 o	← texte()	– un texte gravé	
"TEMPS"	5 o	← texte()	– un texte gravé	
"FINI!"	5 o	← texte()	– un texte gravé	
"POINTS"	6 o	← texte()	– un texte gravé	
"NOUVEAU RECORD!"	15 o	← texte()	– un texte gravé	
"START: TITRE"	12 o	← texte()	– un texte gravé	

Lettres gravées : ACDEFIJLMNOPRSTUV0123456789!:

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

texte()	x 11	127 o sur place, dans changer() + AdresseCase (10 o), EcrireTexte (10 o)
sauver()	x 3	73 o sur place, dans changer(), main()
nombre()	x 6	60 o sur place, dans changer(), jeu() + EcrireNombre (33 o)
sauvegarde()	x 2	41 o sur place, dans main()
bouton()	x 2	24 o sur place, dans appuiStart(), jeu() + LireManette (37 o)
ecran()	x 2	21 o sur place, dans changer() + AttendreVBlank (16 o)
images()	x 2	4 o sur place, dans changer(), jeu()
image()	x 1	3 o sur place, dans main() + AttendreImage (28 o)

Tes fonctions appelées :

changer()	x 4	26 o sur place, dans titre(), jeu(), perdu(), main()
appuiStart()	x 2	6 o sur place, dans titre(), perdu()
titre()	x 1	3 o sur place, dans main()
jeu()	x 1	3 o sur place, dans main()
perdu()	x 1	3 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : « LE JEU DU A », avec un RECORD qui reste même quand on
// éteint la console.
//
// Ce qui est nouveau : la mémoire de la cartouche (elle a une pile).
//  sauver(numero, valeur) : écrit valeur dans la case « numero » de la cartouche.
//  sauvegarde(numero)      : relit cette case, même des mois plus tard.
//  La toute première fois, cette mémoire contient n'importe quoi. On écrit donc
//  une MARQUE (42) dans la case 0 : si on ne la retrouve pas, la mémoire est neuve.

#include <bouton>      // lit un bouton de la manette
#include <ecran>       // éteint ou rallume l'écran
#include <texte>       // écrit un texte à l'écran
#include <nombre>      // écrit un nombre en chiffres
#include <sauver>       // garde un nombre dans la cartouche, même éteinte
#include <sauvegarde>   // relit un nombre gardé dans la cartouche
```

2. DES CONSTANTES NOMMÉES — ETAT

```
enum Etat { TITRE, JEU, PERDU }; // Les trois écrans.
```

3. UNE CONSTANTE — MARQUE

```
const uint8_t MARQUE = 42; // « cette cartouche a déjà servi à ce jeu »
```

4. UNE VARIABLE — ETAT

```
Etat etat = TITRE;
```

5. UNE VARIABLE — SCORE

```
uint8_t score = 0;
```

6. UNE VARIABLE — RECORD

```
uint8_t record = 0;          // Le meilleur score, relu dans la cartouche au démarrage.
```

7. UNE VARIABLE — RESTE

```
uint8_t reste = 5;          // Les secondes de la partie.
```

8. UNE VARIABLE — TOP

```
uint8_t top = 0;            // images() au début de la seconde en cours.
```

9. UNE VARIABLE — AVANTSTART

```
uint8_t avantStart = 0;     // START à l'image d'avant.
```

10. UNE VARIABLE — AVANTA

```
uint8_t avantA = 0;         // A à l'image d'avant.  
  
// 1 à la première image d'un appui sur START (le front).
```

11. UNE FONCTION — APPUISTART()

```
uint8_t appuiStart() {  
    uint8_t s = bouton(START);  
    uint8_t front = s && !avantStart;  
    avantStart = s;  
    return front;  
}  
  
// La seule fonction qui change d'état.
```

12. UNE FONCTION — CHANGER()

```
void changer(Etat nouvel) {
    etat = nouvel;
    ecran(0);
    for (uint8_t l = 4; l < 14; l++) texte(0, l, "          "); // efface lignes 4 à 13

    switch (etat) {
        case TITRE:
            texte(5, 6, "LE JEU DU A");
            texte(3, 8, "RECORD");
            nombre(10, 8, record); // Le record s'affiche au titre.
            texte(4, 11, "START: JOUER");
            break;
        case JEU:
            score = 0; // Tout repart de zéro.
            reste = 5;
            top = images();
            texte(3, 6, "APPUIE SUR A!");
            texte(3, 9, "POINTS");
            nombre(10, 9, score);
            texte(3, 10, "TEMPS");
            nombre(10, 10, reste, 1);
            break;
        case PERDU:
            texte(7, 6, "FINI!");
            texte(3, 9, "POINTS");
            nombre(10, 9, score);
            if (score > record) { // un nouveau record ?
                record = score;
                sauver(1, record); // dans la cartouche (case 1) : il survivra
                texte(2, 11, "NOUVEAU RECORD!");
            } // (On ne sauve QUE s'il est battu : écrire
            // dans la cartouche est lent et l'utilise un peu.)

            texte(4, 13, "START: TITRE");
            break;
    }
    ecran(1);
}
```

13. UNE FONCTION — TITRE()

```
void titre() { // Au titre : START → la partie.
    if (appuiStart()) changer(JEU);
}
```

14. UNE FONCTION — JEU()

```
void jeu() { // Pendant la partie :
    uint8_t a = bouton(A);
    if (a && !avantA) { // un appui sur A = un point
        score++;
        nombre(10, 9, score);
    }
    avantA = a;

    uint8_t ecoule = images() - top;
    if (ecoule >= 60) { // une seconde est passée
        top = top + 60;
        reste--;
        nombre(10, 10, reste, 1);
        if (reste == 0) changer(PERDU); // plus de temps → fin
    }
}
```

15. UNE FONCTION — PERDU()

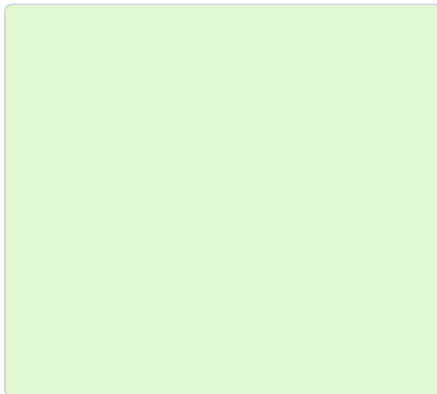
```
void perdu() {                                // À la fin : START → le titre.  
    if (appuiStart()) changer(TITRE);  
}
```

16. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

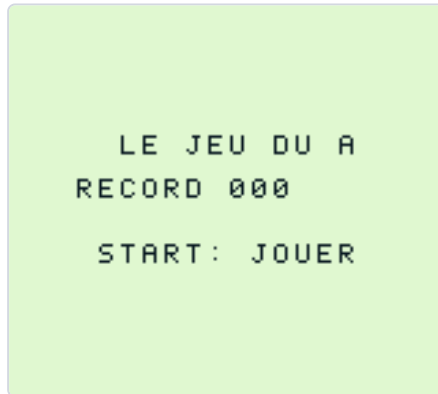
```
int main() {                                // Le jeu commence ici.  
    // La cartouche a-t-elle déjà servi ?  
    if (sauvegarde(0) != MARQUE) {  
        sauver(0, MARQUE);                // non : on la marque  
        sauver(1, 0);                      // et le record part de 0  
    }  
    record = sauvegarde(1);                // on relit le record gardé  
  
    changer(TITRE);                        // On dessine le titre.  
  
    while (true) {                          // La boucle du jeu : elle aiguille selon l'état.  
        image();  
        switch (etat) {  
            case TITRE: titre(); break;  
            case JEU:  jeu();  break;  
            case PERDU: perdu(); break;  
        }  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.

```
LE JEU DU A
RECORD 000

START: JOUER
```

A — l'écran n'a pas bougé

```
APPUIE SUR A!

POINTS 000
TEMPS 5
```

START — l'écran a changé

5. Ce qu'on doit voir

Le titre montre RECORD 000 ; après une partie, NOUVEAU RECORD! ; de retour au titre, le record est là — et il y serait encore après avoir éteint.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la marque 42 est écrite dans la cartouche
- ✓ cinq points : NOUVEAU RECORD!
- ✓ le record est gardé dans la cartouche (5)
- ✓ le titre l'affiche

Ensuite : **120.1. Chapitre 21 — Le son : notes, bruits, airs : ce qui vient** — chapitre 21 : Le son : notes, bruits, airs. · gameboy3 — écrire une cartouche Game Boy en C++